



JAVA - INTERMEDIAIRE

Java

Collections, generics, streams, Optional et fichiers pour du Java pro au quotidien.

DanielCraft - Pack Java / langages

Sommaire

1. [Après les bases : Java intermediaire](#)
 - [Ce que tu vas apprendre](#)
 - [A retenir](#)
2. [Collections : List, Set, Map](#)
 - [Choisir](#)
 - [A retenir](#)
3. [Generics : typer sans caster](#)
 - [Wildcards \(idee\)](#)
 - [A retenir](#)
4. [Interfaces et classes abstraites](#)
 - [Default methods](#)
 - [A retenir](#)
5. [Heritage et polymorphisme](#)
 - [Regles utiles](#)
 - [A retenir](#)
6. [Exceptions : gerees proprement](#)
 - [Bonnes pratiques](#)
 - [A retenir](#)
7. [Streams : pipeline sur les donnees](#)
 - [Operations](#)
 - [A retenir](#)
8. [Optional : finir avec null](#)
 - [Regles](#)
 - [A retenir](#)
9. [Fichiers et NIO.2](#)
 - [Idees utiles](#)
 - [A retenir](#)
10. [Records : donnees immuables](#)
 - [Interet](#)
 - [A retenir](#)
11. [Enums riches](#)
 - [Usages](#)
 - [A retenir](#)
12. [Packages et organisation](#)
 - [Idees](#)
 - [A retenir](#)
13. [Tests \(idee JUnit\)](#)
 - [Pourquoi](#)
 - [A retenir](#)

14. Mini-projet : carnet de taches

- Exigences
- Livrables
- A retenir

15. Ce qu'il faut retenir

- Carte inter Java
- Mindset
- A retenir

16. Atelier : collections

- A retenir

17. Atelier : streams

- A retenir

18. Erreurs classiques

- A retenir

19. Bonnes pratiques

- A retenir

20. Quiz Java inter

- Bareme
- A retenir

21. Bravo !

- Et maintenant ?
- A retenir

Après les bases : Java intermédiaire



Des bases aux outils du quotidien.



Couverture Java Intermédiaire.

Tu connais variables, methodes, classes. Le niveau **intermédiaire**, c'est **structures utiles** et **API moderne** : collections, generics, streams, Optional, fichiers.

> **Astuce DanielCraft** - Inter = écrire du Java que tu reliras dans 6 mois sans rougir.

Ce que tu vas apprendre

- List, Set, Map et generics.
- Interfaces, héritage, polymorphisme.
- Streams, Optional, fichiers.
- Records, enums, mini-projet.

A retenir

- Intermédiaire = outils du quotidien pro.
- Clarté > astuces obscures.

Collections : List, Set, Map



List, Set, Map : choisir la bonne structure.

Les **collections** remplacent les tableaux quand la taille change.

```
List<String> noms = new ArrayList<>();
noms.add("Lea");
noms.add("Max");

Set<Integer> ids = new HashSet<>();
Map<String, Integer> ages = new HashMap<>();
ages.put("Sam", 28);
```

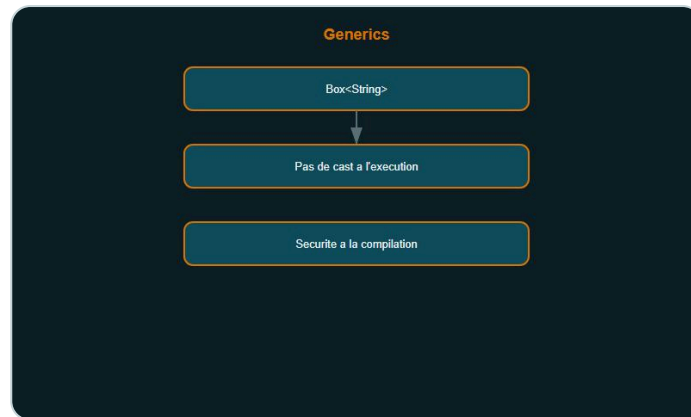
Choisir

Type	Usage
List	Ordre, doublons possibles
Set	Unicité
Map	Cle → valeur

A retenir

- Préférer les interfaces (**List**) aux implementations dans les signatures.

Generics : typer sans caster



Generics : typer sans caster.

Les **generics** evitent les cast et les erreurs a l'execution.

```
Box<String> b = new Box<>();
b.set("ok");
String s = b.get(); // pas de cast
```

Wildcards (idee)

- `List<? extends Number>` : lecture de nombres.
- `List<? super Integer>` : ecriture d'entiers.

> **Piege** - `List<Object>` n'est pas un `List<String>`.

A retenir

- Generics = securite de type a la compilation.

Interfaces et classes abstraites



Interface = contrat, abstraite = code commun.

Une **interface** définit un contrat. Une **classe abstraite** peut partager du code.

```
interface Payable {  
    double montant();  
}  
  
abstract class Personne {  
    String nom;  
    abstract String role();  
}
```

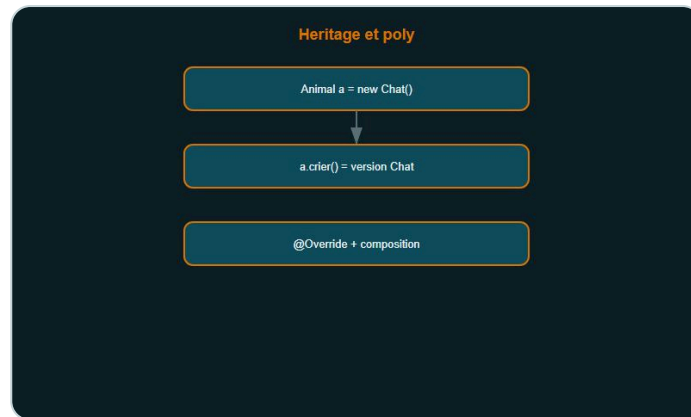
Default methods

Depuis Java 8, une interface peut avoir des méthodes **default**.

A retenir

- Interface = comportement ; abstraite = famille avec code commun.

Heritage et polymorphisme



Polymorphisme : meme appel, comportement adapte.

Le **polymorphisme** : une reference de type parent pointe vers un enfant.

```
Animal a = new Chat();  
a.crier(); // version Chat
```

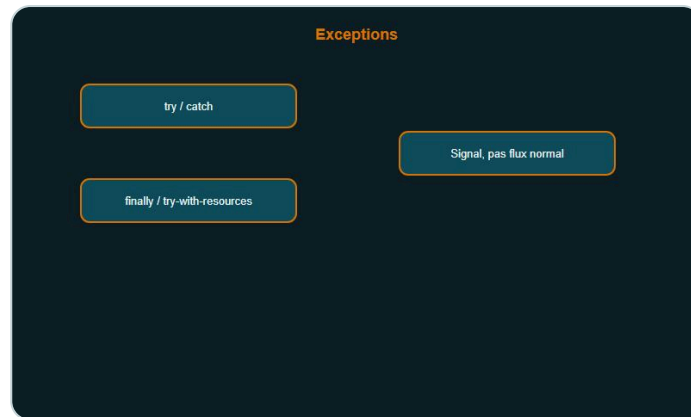
Regles utiles

- `@Override` systematiquement.
- Preferer composition quand l'heritage devient un labyrinthe.
- `final` sur classe / methode pour figer.

A retenir

- Polymorphisme = meme appel, comportement adapte.

Exceptions : gereses proprement



Exceptions : signaler, pas avaler.

```
try {
    Files.readString(Path.of("data.txt"));
} catch (IOException e) {
    System.err.println("Lecture impossible");
} finally {
    // nettoyage si besoin
}
```

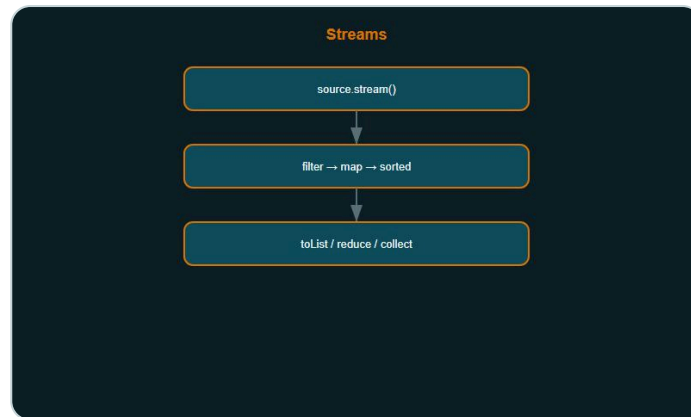
Bonnes pratiques

- Ne pas avaler l'exception (`catch` vide).
- Exceptions **checked** = le compilateur t'oblige a gerer.
- Creer des exceptions metier si le message compte.

A retenir

- Exception = signal, pas flux normal.

Streams : pipeline sur les donnees



Pipeline filter → map → terminal.

```
List<Integer> pairs = nombres.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 2)
    .toList();
```

Operations

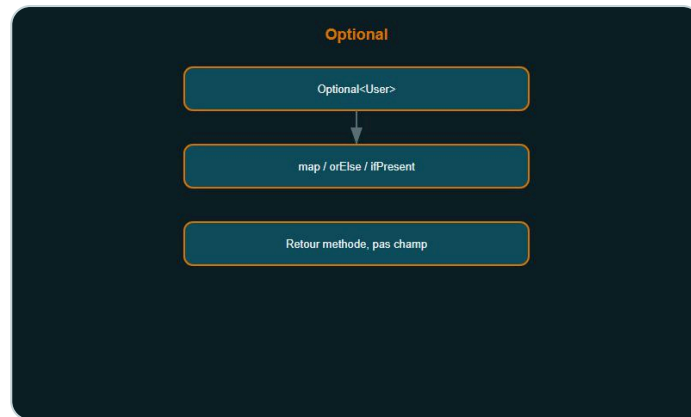
- Intermediaires : `filter`, `map`, `sorted`.
- Terminales : `toList`, `forEach`, `reduce`, `collect`.

> **Astuce DanielCraft** - Un stream court et lisible bat une boucle opaque de 40 lignes.

A retenir

- Stream = declaration du quoi, pas du comment detaille.

Optional : finir avec null



Optional : finir avec null.

```
Optional<User> u = findUser(id);  
String name = u.map(User::getName).orElse("inconnu");
```

Regles

- Retour de methode : OK.
- Champ / parametre : en general non.
- Eviter `get()` sans `isPresent` / `orElse`.

A retenir

- Optional = intention "peut etre absent".

Fichiers et NIO.2



Path + Files : API moderne.

```
Path p = Path.of("notes.txt");
Files.writeString(p, "Bonjour");
String contenu = Files.readString(p);
```

Idees utiles

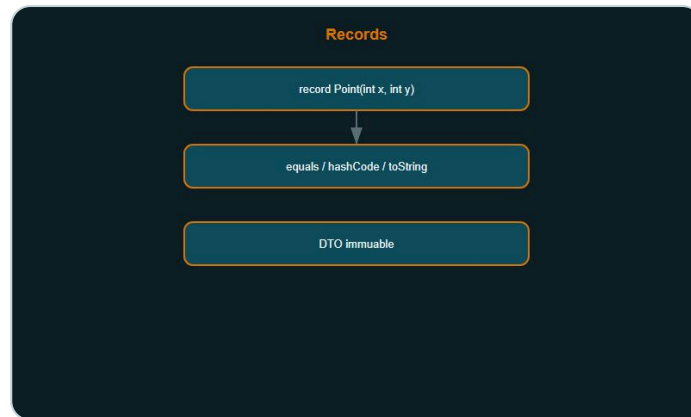
- `Files.lines(path)` + stream pour gros fichiers.
- `try-with-resources` pour fermer automatiquement.

```
try (var reader = Files.newBufferedReader(p)) {
    // ...
}
```

A retenir

- Path + Files = API moderne préférée à File legacy.

Records : donnees immuables



Record : donnees immuables.

```
public record Point(int x, int y) {}  
Point p = new Point(3, 4);  
System.out.println(p.x());
```

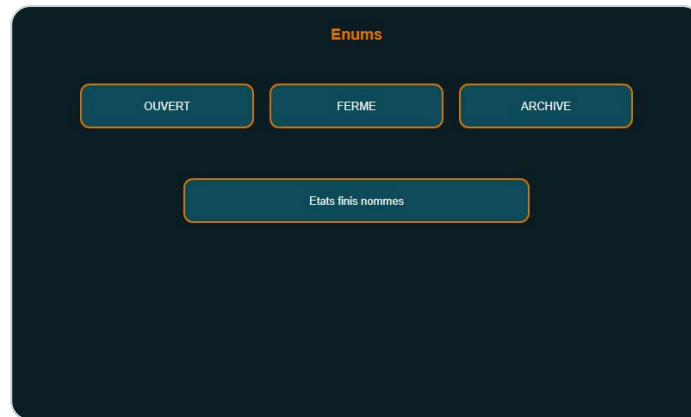
Interet

- Moins de boilerplate (equals, hashCode, toString).
- Ideal DTO / resultats intermediaires.

A retenir

- Record = porteur de donnees, pas logique lourde.

Enums riches



Enum : etats finis nommes.

```
enum Statut {  
    OUVERT, FERME, ARCHIVE;  
  
    boolean estActif() {  
        return this == OUVERT;  
    }  
}
```

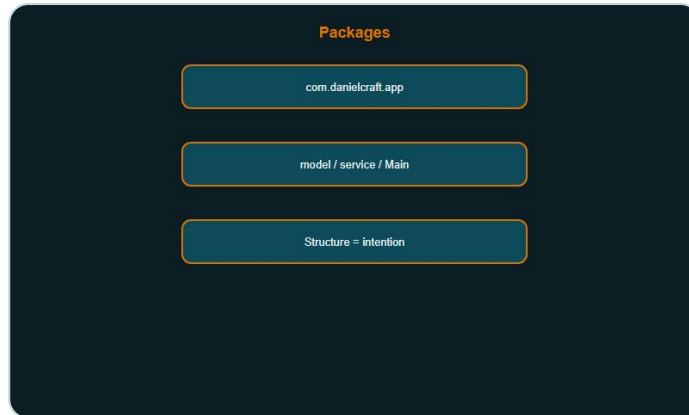
Usages

- Etats finis (workflow).
- Remplacer les constantes String magiques.

A retenir

- Enum = ensemble ferme de valeurs nommees.

Packages et organisation



Packages : structure claire.

```
com.danielcraft.app
  model/
    service/
      Main.java
```

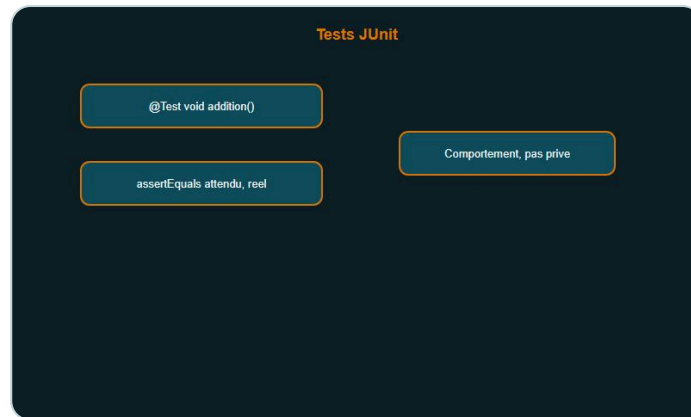
Idees

- Package = intention (`service` , pas `utils2`).
- Visibilite package-private par default pour limiter l'API.
- Modules JPMS : idee avantee (exports explicites).

A retenir

- Bonne structure > gros dossier fourre-tout.

Tests (idée JUnit)



Tests : verifier le comportement.

```
@Test
void addition() {
    assertEquals(4, Calc.add(2, 2));
}
```

Pourquoi

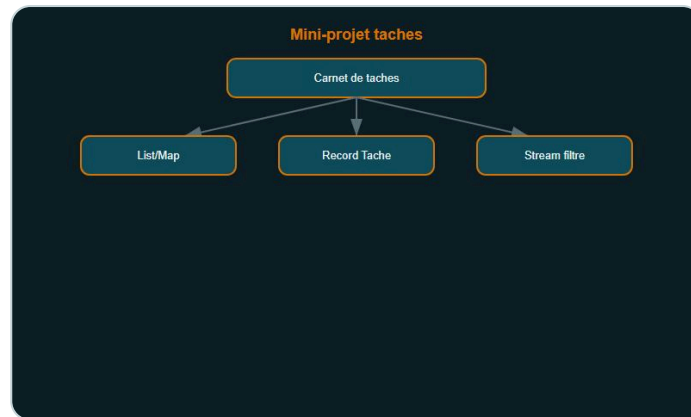
- Regresser moins.
- Documenter le comportement attendu.

> **Astuce DanielCraft** - Un test sur la regle metier bat dix commentaires flous.

A retenir

- Tester le comportement, pas l'implementation privee.

Mini-projet : carnet de taches



Mini-projet carnet de taches.

Objectif : CLI simple de taches (ajouter, lister, terminer).

Exigences

- `List<Tache>` ou `Map<Integer, Tache>`.
- Record `Tache(int id, String titre, boolean done)`.
- Stream pour filtrer les ouvertes.
- Sauvegarde fichier texte optionnelle.

Livrables

1. Classes + Main.
2. 3 tests JUnit sur ajout / filtre.
3. README 10 lignes.

A retenir

- Mini-projet = collections + streams + fichiers.

Ce qu'il faut retenir



Carte inter Java.

Carte inter Java

- Collections + generics.
- Interfaces / poly.
- Streams + Optional.
- Fichiers, records, enums, tests.

Mindset

- API claire, types explicites, peu de null.

A retenir

- Inter Java = boîte à outils du quotidien.

Atelier : collections



Atelier collections.

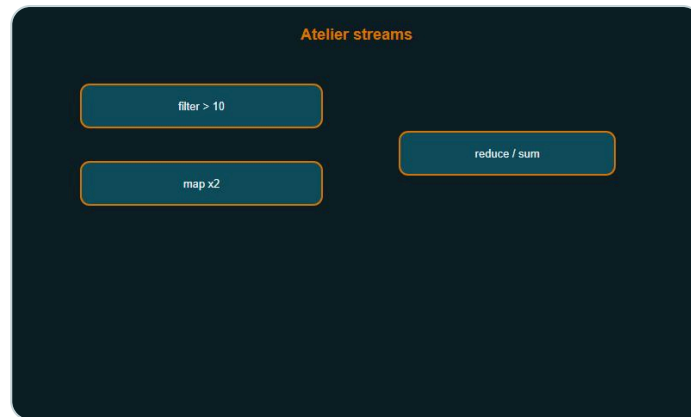
Duree : 20 min.

1. Liste de prenom, supprimer les doublons via **Set** .
2. **Map** prenom → longueur.
3. Afficher les prenom de longueur > 4.

A retenir

- Choisir List / Set / Map selon le besoin.

Atelier : streams



Atelier streams.

Duree : 20 min.

Sur une `List<Integer>` :

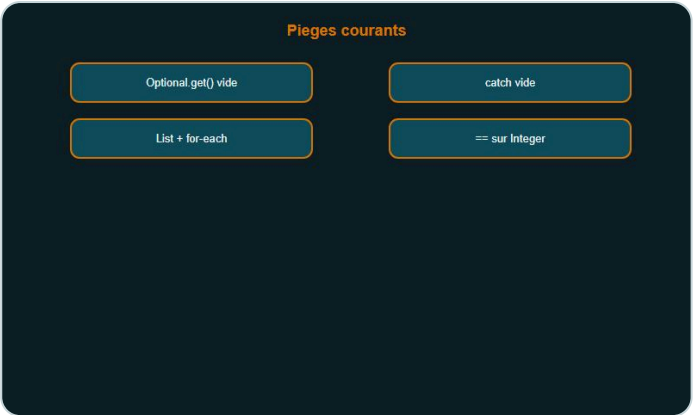
1. Filtrer > 10 .
2. Multiplier par 2.
3. Sommer (`reduce` ou `mapToInt`).

A retenir

- `filter` $\rightarrow$ `map` $\rightarrow$ terminal.

CHAPITRE 18

Erreurs classiques



Pieges courants.

Erreur	Fix
<code>get()</code> sur Optional vide	<code>orElse</code> / <code>ifPresent</code>
Modifier une List pendant for-each	Iterator / <code>removeIf</code>
Catch vide	Logger ou remonter
Heritage a 6 niveaux	Composition
<code>==</code> sur Integer caches	<code>equals</code>

A retenir

- Les pieges courants ont des antidotes simples.

Bonnes pratiques



Discipline pro.

1. Interfaces dans les signatures.
2. Immuabilite quand possible (record, final).
3. Noms metier (`CommandeService` , pas `Manager2`).
4. try-with-resources.
5. Tests sur les regles critiques.

A retenir

- Style pro = predictable.

Quiz Java inter



Quiz Java intermediaire.

1. 1. List vs Set ?
2. 2. A quoi sert un generic ?
3. 3. Difference interface / classe abstraite ?
4. 4. Operation terminale d'un stream ?
5. 5. Optional : bon usage ?
6. 6. Avantage d'un record ?
7. 7. try-with-resources sert a ?
8. 8. Pourquoi `@Override` ?
9. 9. Map : que stocke-t-elle ?
10. 10. Un test unitaire verifie quoi ?

Bareme

- 8/10+ : pret pour la suite (Spring, etc.).
- Moins : relire collections + streams.

A retenir

- Quiz = diagnostic, pas sanction.

FINAL

Bravo !



Bravo. Du Java clair, c'est du Java durable.

Tu as termine **Java - Intermediaire** : collections, generics, streams, Optional, fichiers, records.

Et maintenant ?

- Refactorise un vieux tableau en List/Map.
- Ajoute 3 tests sur une regle metier.
- Explore Spring plus tard, avec ces bases solides.

> **DanielCraft** - Du Java clair, c'est du Java durable.

A retenir

- Continue a lire du code et a en ecrire petit a petit.

Tu structures. Tu streams. Tu livres.